

Laboratory Work 1

Task 1.1:

Relation A — Employee(EmpID, SSN, Email, Phone, Name, Department, Salary)

1. Six or more superkeys:

- {EmpID}
- {SSN}
- {Email}
- {EmpID, Name}
- {SSN, Department}
- {Email, Salary}
- {EmpID, SSN, Email, Phone, Name, Department, Salary} (the entire relation, always a superkey)

2. Candidate keys:

{EmpID}, {SSN}, and {Email} are each candidate keys. Each is a single attribute that uniquely identifies a tuple, and — being atomic — none can be reduced further, so minimality holds automatically. Phone is NOT a candidate key: nothing in the requirements guarantees a phone number is never shared (e.g., a shared department line), so uniqueness cannot be assumed even though the three sample rows happen to show distinct values. Name, Department, and Salary are clearly not unique.

3. Recommended primary key: EmpID.

Reasoning: SSN is highly sensitive personal data, so using it as a primary key (and propagating it as a foreign key into other tables) creates unnecessary privacy and legal exposure. Email can change when an employee changes teams, gets married, or the company changes its domain naming convention — a primary key should be stable over time. EmpID is a system-generated surrogate key: it is stable, non-sensitive, and exists purely to identify the row, which makes it the safest and most practical choice.

4. Can two employees share the same phone number?

Yes. Nothing in the schema or the business scenario forbids it — for example, two employees who share a desk or an office extension could have identical Phone values. The fact that the three sample rows show different phone numbers is just a property of that particular sample; it does not amount to a constraint that phone numbers must be unique. Since a key must guarantee uniqueness for every possible instance of the relation (not merely the sample shown), Phone cannot be treated as a candidate key.

Relation B — Registration(StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

1. Minimum attributes needed for the primary key:

Primary key = {StudentID, CourseCode, Section, Semester, Year}

2. Why each attribute is necessary:

- StudentID — distinguishes which student the registration row belongs to.
- CourseCode — distinguishes which course was taken (a student can be registered in several different courses).
- Section — a course can be split into multiple sections in the same semester; without it, two rows for the same course/semester but different sections would collide.

- Semester (e.g., Fall/Spring) — the business rule states a student may retake the same course in a different semester, so Semester is needed to tell those attempts apart.
- Year — Semester names repeat every year ("Fall" happens every year), so Year is required to distinguish, e.g., Fall 2025 from Fall 2026.

Removing any one of these five attributes would allow two genuinely different registrations (e.g., the same student retaking the same course in a later year) to collide on the remaining attributes — so all five are necessary, and together they are sufficient because the business rules guarantee no two rows can share the same combination.

3. Additional candidate keys:

None. Grade and Credits are descriptive/dependent attributes (the outcome and value of a specific registration), not identifying attributes, so they cannot participate in an alternate key. This relation has exactly one candidate key.

Task 1.2: Foreign Key Design

Given tables: Student, Professor, Course, Department, Enrollment. Primary keys are underlined in the schema listing below (shown in bold since Word bullet lists cannot render underlines on individual attributes here); foreign keys are annotated with →.

Table	Primary Key	Foreign Key(s)	References
Student	StudentID	AdvisorID	Professor.ProfID
Professor	ProfID	Department	Department.DeptCode
Course	CourseID	DepartmentCode	Department.DeptCode
Department	DeptCode	ChairID	Professor.ProfID
Enrollment	StudentID + CourseID + Semester (composite)	StudentID, CourseID	Student.StudentID, Course.CourseID

Notes on the design:

- Enrollment is a classic associative (junction) table resolving the many-to-many relationship between Student and Course; its primary key is the composite (StudentID, CourseID, Semester), since a student can retake a course in a later semester.
- Professor.Department → Department.DeptCode and Department.ChairID → Professor.ProfID form a mutual (circular) reference: a professor belongs to a department, and a department has a chair who is a professor. This is common in real schemas and is normally handled by making one of the two foreign keys nullable (e.g., ChairID can be NULL until a chair is assigned) or by deferring the constraint check until both rows exist.

Task 2.1: Hospital Management System

1–2. Entities and Attributes

Entity	Type	Key Attribute(s)	Other Attributes (classification)
Patient	Strong	PatientID (simple)	Name (simple), BirthDate (simple), Address (composite: Street, City, State, Zip), PhoneNumbers (multi-valued), InsuranceInfo (composite: Provider, PolicyNumber), Age (derived, from BirthDate)

Entity	Type	Key Attribute(s)	Other Attributes (classification)
Doctor	Strong	DoctorID (simple)	Name (simple), Specializations (multi-valued), PhoneNumber (simple), OfficeLocation (simple)
Department	Strong	DeptCode (simple)	DeptName (simple), Location (simple)
Room	Weak	RoomNumber (partial key)	Full identifying key is (DeptCode, RoomNumber), since "Room 101 in Cardiology" ≠ "Room 101 in Neurology"
Appointment	Strong (associative)	AppointmentID (simple, surrogate)	DateTime (simple), Purpose (simple), Notes (simple)
Prescription	Strong (associative)	PrescriptionID (simple, surrogate)	MedicationName (simple), Dosage (simple), Instructions (simple)

Appointment and Prescription are modeled as regular entities (rather than plain relationships) because each carries its own descriptive attributes beyond the two entities it connects — this is the standard approach whenever a many-to-many association needs to store data about the association itself.

3. Relationships and Cardinalities

- Patient HAS Appointment — 1:N (a patient may have many appointments; each appointment belongs to exactly one patient)
- Doctor ATTENDS Appointment — 1:N (a doctor sees many appointments; each appointment is staffed by exactly one doctor)
- Patient RECEIVES Prescription — 1:N
- Doctor WRITES Prescription — 1:N
- Department HAS_ROOM Room — 1:N, identifying relationship (Room is weak and cannot exist without its owning Department; total participation on the Room side)

4. ER Diagram

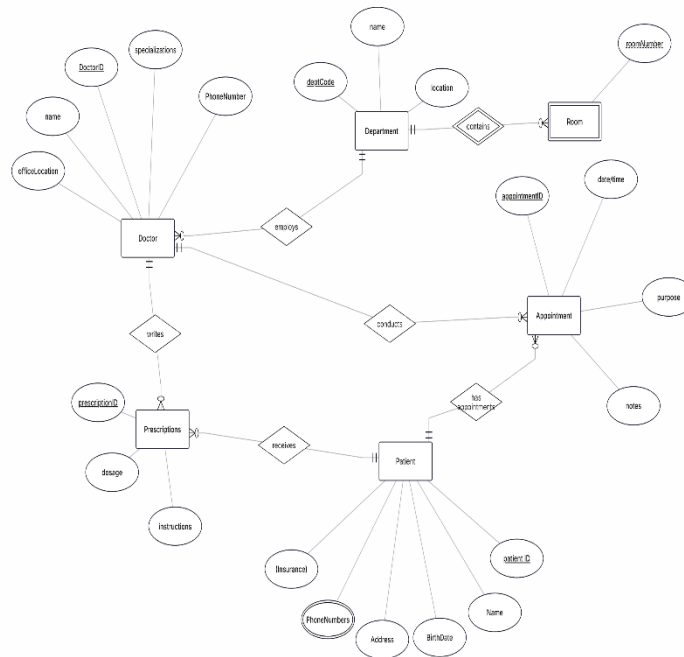


Figure 1 — Hospital Management System ER diagram (Chen notation, simplified).

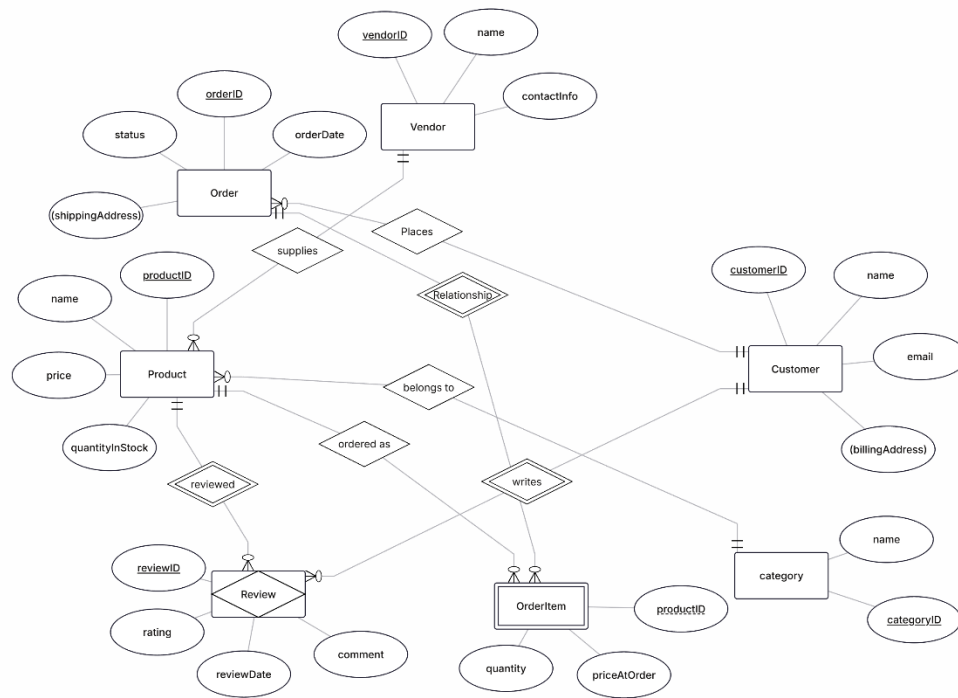
5. Primary Keys

PatientID (Patient), DoctorID (Doctor), DeptCode (Department), (DeptCode, RoomNumber) (Room — weak), AppointmentID (Appointment), PrescriptionID (Prescription).

Task 2.2: E-commerce Platform

Entities and Key Attributes

Entity	Type	Key	Notes
Customer	Strong	CustomerID	Name, Email, BillingAddress (composite)
ShippingAddress	Weak (owned by Customer)	(CustomerID, AddressID)	Street, City, State, Zip — a customer can store several shipping addresses, reused across orders
Order	Strong	OrderID	OrderDate, Status
OrderItem	Weak (owned by Order)	(OrderID, ProductID)	Quantity, PriceAtOrder — resolves the M:N between Order and Product
Product	Strong	ProductID	Name, Price
Category	Strong	CategoryID	CategoryName
Vendor	Strong	VendorID	VendorName, ContactInfo
Review	Strong (associative)	ReviewID	Rating, Comment, Date — resolves the M:N between Customer and Product
Inventory	Weak (1:1 owned by Product)	ProductID	StockLevel, Warehouse



ER Diagram

Figure 2 — E-commerce Platform ER diagram.

2. Weak Entity Justification

ORDER_ITEM is a weak entity. A line item ("3 units of Product X at \$19.99") has no meaning or independent existence outside the specific Order it belongs to — it cannot be identified by ProductID alone (the same product can appear as a line item on many different orders), so it borrows OrderID from its owner to form its full identifying key: (OrderID, ProductID). If the owning Order is deleted, its OrderItem rows are deleted with it (existence dependency), which is the defining property of a weak entity.

3. Many-to-Many Relationship Requiring Attributes

ORDER ↔ PRODUCT, resolved through ORDER_ITEM: an order can contain many products, and a product can appear on many orders, and the relationship itself needs to carry Quantity and PriceAtOrder (the price must be captured at the time of purchase, since the Product's current price may later change). A second example is CUSTOMER ↔ PRODUCT, resolved through REVIEW, which carries Rating, Comment, and Date.

Task 4.1: Denormalized Table Analysis

StudentProject(StudentID, StudentName, StudentMajor, ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, Role, HoursWorked, StartDate, EndDate)

Assumption: a student can work on several projects and a project can have several students (each with their own Role), so the natural primary key is the composite (StudentID, ProjectID). Each project has exactly one supervisor.

1. Functional Dependencies

- StudentID → StudentName, StudentMajor
- ProjectID → ProjectTitle, ProjectType, SupervisorID
- SupervisorID → SupervisorName, SupervisorDept
- (StudentID, ProjectID) → Role, HoursWorked, StartDate, EndDate

2. Problems

Redundancy: StudentName and StudentMajor are repeated on every row for every project that student works on. ProjectTitle, ProjectType, SupervisorID, SupervisorName, and SupervisorDept are repeated on every row for every student assigned to that project.

Anomalies:

- Update anomaly: if a student changes major, every row containing that StudentID must be updated; missing even one causes inconsistent data.
- Insert anomaly: a new project cannot be recorded until at least one student has been assigned to it, and a new student cannot be recorded until assigned to a project — the entities are not independent of each other.
- Delete anomaly: if the only student assigned to a project is removed from the table, all information about that project (Title, Type, Supervisor) is lost along with the student's row.

3. First Normal Form (1NF)

No 1NF violations are present: every attribute holds a single atomic value (no repeating groups or multi-valued fields such as "multiple roles" packed into one cell), so the table is already in 1NF.

4. Second Normal Form (2NF)

Primary key: (StudentID, ProjectID).

Partial dependencies (attributes depending on only part of the key):

- StudentID → StudentName, StudentMajor (depends only on StudentID, not on ProjectID)
- ProjectID → ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept (depends only on ProjectID, not on StudentID)

2NF decomposition:

Table	Attributes	Primary Key
Student	StudentID, StudentName, StudentMajor	StudentID
Project	ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept	ProjectID
Assignment	StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate	(StudentID, ProjectID)

5. Third Normal Form (3NF)

Transitive dependency: inside the Project table above, ProjectID → SupervisorID → SupervisorName, SupervisorDept. SupervisorName and SupervisorDept depend on SupervisorID, not directly on ProjectID — a textbook transitive dependency that violates 3NF.

Final 3NF decomposition:

Table	Attributes	Primary Key	Foreign Key(s)
Student	StudentID, StudentName, StudentMajor	StudentID	—

Table	Attributes	Primary Key	Foreign Key(s)
Supervisor	SupervisorID, SupervisorName, SupervisorDept	SupervisorID	—
Project	ProjectID, ProjectTitle, ProjectType, SupervisorID	ProjectID	SupervisorID → Supervisor
Assignment	StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate	(StudentID, ProjectID)	StudentID → Student, ProjectID → Project

Task 4.2: Advanced Normalization

CourseSchedule(StudentID, StudentMajor, CourseID, CourseName, InstructorID, InstructorName, TimeSlot, Room, Building)

1–2. Functional Dependencies and Primary Key

Translating the business rules into FDs:

- StudentID → StudentMajor ("each student has exactly one major")
- CourseID → CourseName ("each course has a fixed name")
- InstructorID → InstructorName ("each instructor has exactly one name")
- Room → Building ("rooms are unique across campus", so a room number alone fixes its building)
- (InstructorID, TimeSlot) → Room, CourseID (an instructor can only be in one room teaching one course at a given time slot)
- (Room, TimeSlot) → InstructorID, CourseID (a room can only host one class at a given time slot)
- (CourseID, TimeSlot) → InstructorID, Room ("each course section is taught by one instructor at one time in one room")

Primary key (the "tricky" part): (StudentID, CourseID, TimeSlot).

Because the scheduling rules make CourseID+TimeSlot, InstructorID+TimeSlot, and Room+TimeSlot mutually determine one another (they all identify the same class meeting), there are actually three overlapping candidate keys for the "which class meeting" part of the row: (StudentID, CourseID, TimeSlot), (StudentID, InstructorID, TimeSlot), and (StudentID, Room, TimeSlot). Any one of them is a valid primary key; (StudentID, CourseID, TimeSlot) is simply the most natural choice since CourseID is what a student actually registers for.

3. Is the table in BCNF?

No. BCNF requires that the left-hand side of every non-trivial FD be a superkey of the whole relation. Every determinant found above — StudentID, CourseID, InstructorID, Room, (InstructorID,TimeSlot), (Room,TimeSlot), (CourseID,TimeSlot) — determines only some of the attributes and never determines StudentID (many students can sit in the same class), so none of them is a superkey of the full table. Every one of these FDs therefore violates BCNF.

4. BCNF Decomposition

Decompose repeatedly on a violating FD, keeping the determinant in both resulting relations, until every remaining relation is in BCNF:

- Split off CourseID → CourseName: ⇒ Course(CourseID, CourseName)
- Split off InstructorID → InstructorName: ⇒ Instructor(InstructorID, InstructorName)
- Split off Room → Building: ⇒ RoomInfo(Room, Building)
- Split off StudentID → StudentMajor: ⇒ StudentInfo(StudentID, StudentMajor)

- What remains, (StudentID, CourseID, InstructorID, TimeSlot, Room), still violates BCNF via (CourseID, TimeSlot) → InstructorID, Room. Split it into ClassSection(CourseID, TimeSlot, InstructorID, Room) and Enrollment(StudentID, CourseID, TimeSlot).

Final BCNF schema:

Table	Attributes	Primary Key	Notes
Course	CourseID, CourseName	CourseID	—
Instructor	InstructorID, InstructorName	InstructorID	—
RoomInfo	Room, Building	Room	—
StudentInfo	StudentID, StudentMajor	StudentID	—
ClassSection	CourseID, TimeSlot, InstructorID, Room	(CourseID, TimeSlot)	Alternate keys: (InstructorID, TimeSlot), (Room, TimeSlot). FKs to Course, Instructor, RoomInfo.
Enrollment	StudentID, CourseID, TimeSlot	(StudentID, CourseID, TimeSlot)	FK to StudentInfo and to ClassSection (CourseID, TimeSlot)

5. Loss of Information in the Decomposition

None, in the lossless-join sense: at every step the split was performed on a functional dependency $X \rightarrow Y$, and X (the shared attribute set) was kept in both resulting tables, which is precisely the condition that guarantees a lossless-join decomposition — joining the pieces back together on their shared keys reconstructs the original table exactly. Dependency preservation is also maintained: the three mutually-determining scheduling FDs — $(\text{CourseID}, \text{TimeSlot}) \rightarrow (\text{InstructorID}, \text{Room})$, $(\text{InstructorID}, \text{TimeSlot}) \rightarrow (\text{CourseID}, \text{Room})$, and $(\text{Room}, \text{TimeSlot}) \rightarrow (\text{CourseID}, \text{InstructorID})$ — are all still checkable directly within the single ClassSection table, since none of those four attributes were separated from one another.

Part 5: Design Challenge

Task 5.1: Student Clubs and Organizations

1. Entities and Relationships

- STUDENT — ClubMember, participates via MEMBERSHIP and ATTENDANCE
- CLUB — has a Budget; ADVISED by one FACULTY_ADVISOR; HOSTS Events; INCURS Expenses
- FACULTY_ADVISOR — a faculty member can advise several clubs (1:N)
- MEMBERSHIP — associative entity resolving the M:N between Student and Club, carrying JoinDate and Position (President, Treasurer, Secretary, Member, ...)
- EVENT — belongs to one Club; has one ROOM_RESERVATION (weak, 1:1); tracked via ATTENDANCE
- ATTENDANCE — associative entity resolving the M:N between Student and Event, carrying AttendanceStatus
- ROOM_RESERVATION — weak entity, existence-dependent on its Event (RoomName, Date, TimeSlot)
- EXPENSE — weak entity, existence-dependent on its Club (Amount, Date, Description)

ER Diagram

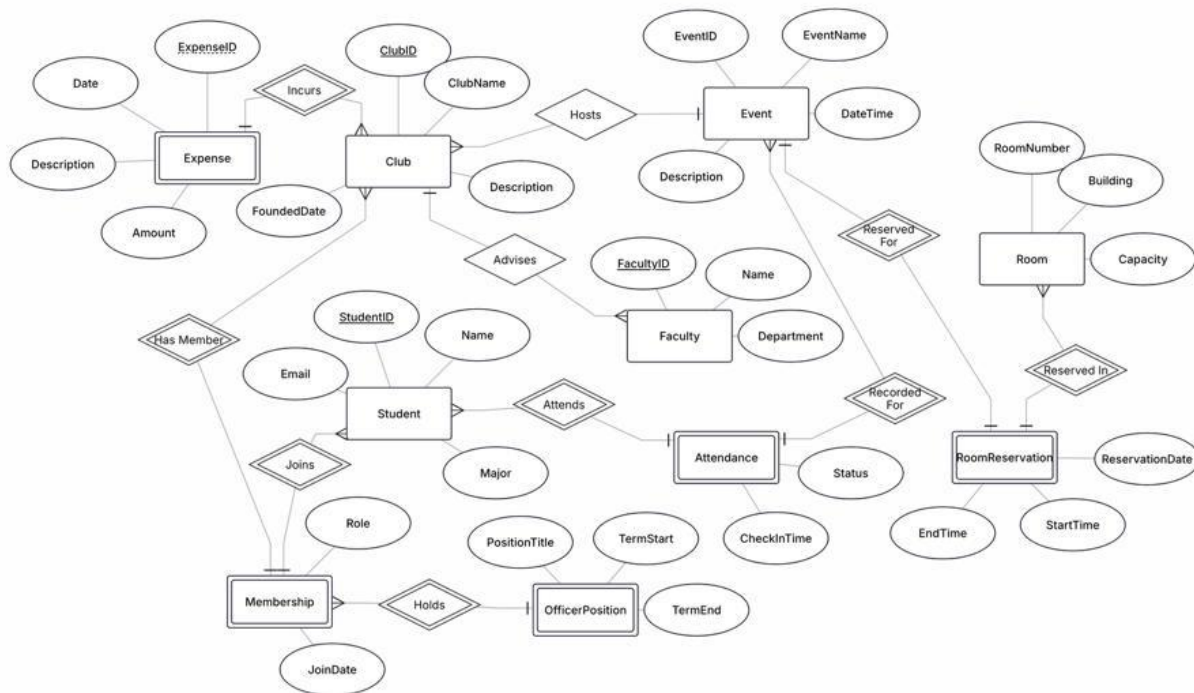


Figure 3 — Student Clubs & Organizations ER diagram.

2. Normalized Relational Schema

Table	Attributes	Primary Key	Foreign Key(s)
Student	StudentID, Name, Email, Major	StudentID	—
FacultyAdvisor	FacultyID, Name, Department	FacultyID	—
Club	ClubID, ClubName, Description, Budget, FacultyID	ClubID	FacultyID → FacultyAdvisor
Membership	StudentID, ClubID, JoinDate, Position	(StudentID, ClubID)	StudentID → Student, ClubID → Club
Event	EventID, ClubID, EventName, EventDate, EventTime, Description	EventID	ClubID → Club
Attendance	StudentID, EventID, AttendanceStatus	(StudentID, EventID)	StudentID → Student, EventID → Event
RoomReservation	EventID, RoomName, ReservedDate, TimeSlot	EventID	EventID → Event (also FK, since 1:1 weak)
Expense	ClubID, ExpenseNum, Amount, ExpenseDate, Description	(ClubID, ExpenseNum)	ClubID → Club

3. A Design Decision With Multiple Valid Options

Decision: how to model officer positions (president, treasurer, secretary, ...).

Option A (chosen): store Position as a plain attribute on the Membership table. This is simple and sufficient because the requirements only ask to know who currently holds which position in each club — they do not ask for a history of past officers.

Option B (rejected for now): a separate OfficerTerm table (ClubID, PositionTitle, StudentID, TermStart, TermEnd), which would support querying who held an office in a previous year.

I chose Option A because the stated requirements describe only the current state of club membership, and Option B would add a table and joins that nothing in the requirements calls for. If the club needed to track officer history over multiple years, Option B would become the correct choice.

4. Example Queries (in English)

1. Find all students who currently hold the position of Treasurer in any club.
2. List all events scheduled for the Chess Club in the next 30 days, along with their reserved rooms and time slots.
3. Calculate the total expenses recorded for each club during the current semester.